計算物理学II

第5回
数値計算プログラミング入門（2）：
ループ, 条件分岐

秋山 進一郎

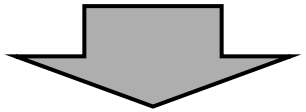
2026年11月6日

授業日の確認

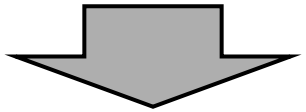
- 全10回
  - 第1回：10月2日（金）
  - 第2回：10月9日（金）
  - 第3回：10月16日（金）
  - 第4回：10月23日（金）
  - **第5回：11月6日（金）**
  - 第6回：11月13日（金）
  - 第7回：11月20日（金）
  - 第8回：12月4日（金）
  - 第9回：12月11日（金）
  - 第10回：12月18日（金）

毎回の授業の流れ

“compphy2”に移動する



“git pull origin main”で演習資料を入手



“lecture5”の中の資料を開いて演習開始

本日の演習内容

- Pythonを通して, 数値プログラミングの基礎的事項を習得する
  - ループ, 条件分岐を使った簡単なプログラムが書けるようになる
  - Pythonのオフサイドルールを理解する
  - プログラムの汎用性, 可読性の効能を理解する

演習用のディレクトリを準備

- まずは今日の演習用ディレクトリをcompphys2の中に作りましょう
  - pathは「ユーザー名@」以下に書いてありますが, 復習を兼ねてpwdで自分の居場所を確認しておきます

```
akiyama@~/compphys2$ pwd
/Users/akiyama/compphys2
```

  - compphys2の中に“work5”というディレクトリを作ります
    - 作ったらlsで確認しましょう

```
akiyama@~/compphys2$ mkdir work5
akiyama@~/compphys2$ ls
README.md lecture1  lecture3  lecture5  vnc       work2     work5
latex     lecture2  lecture4  python    work1     work3
```

Pythonスクリプトファイル

・Pythonインタプリタには二種類ある

① 対話モード（第4回の演習で使っていたモード）

② Pythonスクリプトファイルを使う方法

・②の方法では, スクリプトファイルのパスを引数にして, python3コマンドを実行する

・Pythonスクリプトファイルの拡張子は.py

・今日は②の方法を使っていきます

forによる繰り返し処理 1/3

- 繰り返し処理はプログラミングの基本

- まず, work5の中に, exercise_for.pyというファイルを作りましょう

```
akiyama@~/compphys2$ cd work5
akiyama@~/compphys2/work5$ touch exercise_for.py
```

- 続いて, work5ディレクトリごと, VS Codeで開きましょう

```
akiyama@~/compphys2/work5$ code .
```

- VS Codeが制限モードで開いている人は, 画面左下の
  青い「制限モード/Restricted Mode」を押し,
  青い「信頼する/Trust」を選択しておきましょう



forによる繰り返し処理 2/3

・先ほど用意したexercise_for.pyに以下のようなプログラムを書こう

```
1    for i in range(10):
2        print(i)
```

・このプログラムでは, iの値を0から9まで変化させながら, print(i)が実行される

・プログラムが書けたら保存し, 以下のようにしてterminalから実行してみよう

```
akiyama@~/compphys2/work5$ python3 exercise_for.py
0
1
2
3
4
5
6
7
8
9
```

forによる繰り返し処理 3/3

- forによる繰り返し処理はループと呼ばれる
- このプログラムにおけるiのように, ループの中で値が変化しながら繰り返し実行される変数のことをループカウンタという
- Pythonでは, for文の最後に「：」が必要であり, 「：」の後にコードブロックを伴う
  - for文では「：」に続くコードブロックの中身が繰り返し実行される
  - コードブロックは何行でも良いが, 同じブロックは同じ幅のインデントでないとダメ
    ⇒ 「オフサイドルール」と呼ばれる

インデントは「Tabキー1回」or 「スペース4つ」が推奨
エディタが自動的に入れてくれるかもしれません

- なお, Pythonのオフセット(基準値)は0であることに注意

1~19までの奇数を出力させてみよう

・先ほどのexercise_for.pyを次のように編集してみましょう

```
1    for i in range(10):
2        j = 2*i + 1
3        print(j)
```

・このプログラムでは, iの値を0から9まで変化させながら, 2*i+1を計算し, その結果で変数jを更新した後, print(j)の実行を繰り返します

・プログラムが書けたら保存し, 先ほどの場合と同じようにterminalから実行しましょう

Pythonではコード上のインデント(空白)が意味を持つ

- 今度は, exercise_for.pyの3行目のインデントを消してみましょう

```
1    for i in range(10):
2        j = 2*i + 1
3    print(j)
```

- このプログラムでも, iの値を0から9まで変化させながら, 2*i+1を計算し, その結果で変数jを更新することを繰り返します

- ただし, オフサイドルールにより, 3行目のprint(j)はfor文の外にあり, 繰り返されません

- プログラムが書けたら保存し, terminalから実行しましょう

  - 実行すると「19」だけ出力され, 前のページとは異なる出力を与えるプログラムになっています ⟹ Pythonではインデントがプログラム上で意味を持ちます

プログラムは上の行から順番に実行される

- 今まで書いた1~3行目のプログラムの下に, 新しく次のようなプログラムを書きましょう

- 書けたら保存し, 実行しましょう

- 先ほどの書いたプログラムの出力19に
  つづいて, 0~18の偶数が出力されます

- このように, スクリプトに書いたプログラムは上から下へと順番に実行されます

```
1    for i in range(10):
2        j = 2*i + 1
3    print(j)
4
5    for i in range(10):
6        j = 2*i
7        print(j)
```

- Pythonでは, オフサイドルールにより, インデントが意味を持ちますが, 空行は意味を持ちません(4行目の空行は単にプログラムを読みやすくするために入れています)

ループカウンタはなんでも良い

- ここまで, ループカウンタには毎回iという変数を使ってきましたが, ループカウンタに使う変数はなんでもOKです

- 例えば, 4行目と5行目で使っているループカウンタを好きな変数(文字)に変えてみましょう

- ループカウンタを書き換えたら, 変更を保存して, 実行してみましょう

- 出力結果は, 依然と全く同じはずです

```
1    for i in range(10):
2        j = 2*i + 1
3    print(j)
4
5    for loopcounter in range(10):
6        j = 2*loopcounter
7        print(j)
```

- このように, for文で使うループカウンタはなんでも良いです

ループカウンタを使わないループ

- 単に同じ処理を複数回繰り返したい場合, ループカウンタを使わずにfor文を書くこともできます

- 例として, Hello World!を10回表示するプログラムを書いてみましょう

- exercise_for.pyの9行目以下に,
  次のようなプログラムを書いてみましょう

```
 9    for _ in range(10):
10        print("Hello World!")
```

- Pythonでは, コードブロックでループカウンタを使う処理が不要な場合, ループカウンタに文字を割り振る必要がなく, 単に「 _ 」と書くことができます

  - プログラムが書けたら, 保存をし, 実行してみましょう

- なお, ループカウンタを使って書いても何の問題もありません

  - 9行目の「 _ 」を好きなループカウンタに置き換えても結果は変わりません

ループによる和の計算

・今学んだfor文の典型的な使い方として, 和の計算が挙げられます

・work5の中に, 好きな名前で.pyファイルを用意し, 次のプログラムを書いてみましょう

```
1    total = 0
2    for i in range(10):
3        total = total + i
4    print(total)
```

・このプログラムは, 0から9までの整数の和を計算します(答えは45)

  ・ポイントは, 0で初期化した変数(total)を用意し, 3行目のようにtotalをtotal+iの結果で繰り返し更新する点です

・プログラムが書けたら, 保存をし, 実行しましょう

複合代入演算子

- 前回の演習で学んだように，「＝」は数学の等号ではなく代入演算なので，このプログラムの3行目のような書き方は，プログラミングでは頻出です

- そのため，Pythonでは専用の代入演算子「+=」(複合代入演算子)が用意されています

  - 先ほどの3行目は，複合代入演算子+=を使うと，4行目のように書くことができます

  - 先ほど書いたプログラムを+=を使って下のように書き換え，結果が変わらないことを確かめましょう

```
1    total = 0
2    for i in range(10):
3        #total = total + i
4        total += i
5    print(total)
```

コメントアウト

・なお, 先ほどのプログラムでは3行目の行頭には「#」をつけ, コメントアウトをしました

・コメントアウトされたプログラム(行頭に「#」をつけた部分)は実行されません

・今の例のように, コメントアウトを使うことで, プログラムを書き換えた時に, 書き換えの前後で結果が変わっていないかどうかの確認をスムーズに行えます

・この他にも, コメントアウトによって,プログラム上にメモ書きを残すことができるので, プログラム開発時に重宝することがあります

```
1    total = 0
2    for i in range(10):
3        #total = total + i
4        total += i
5    print(total)
```

ループによる数列の和の計算

- ここまで学んだことを使うだけで, 数列の和を求めるプログラムを書くことができます
- 例えば, 初項$a_0 = 3$, 公差$d = 2$の等比数列$a_n = a_0 + n*d$について,
  初項(n=0)から第10項(n=9)までの和を求めましょう
- work5の中に, 好きな名前で.pyファイルを用意し, for文を使ってプログラムを書いてみましょう
  - まずは自力でプログラムを書いてみましょう
  - 答えは120です
  - 次ページ以降で, この問題を解くためのプログラムの例を説明していきます

解答例

・例えば, 次のようなプログラムが書けたでしょうか

```
1    total = 0
2    for i in range(10):
3        total += 3 + i*2
4    print(total)
```

・このようなプログラムが書ければ, 百点満点です

汎用性・可読性を高める 1/2

- 現時点では, 前ページのようなプログラムが自力で書けていれば何の問題もありません

- しかし, ここでは敢えてもう一歩踏み込んで考えたいと思います

- 数値計算のプログラムを開発する際には, 生の数値を直接書き込んで処理を記述する(ハードコーディングする)よりも, 変数を積極的に活用することで, より汎用性が高く, 読みやすいプログラムを作成することができます

  - このような考え方は, 次回演習で学ぶ「関数」を設計する際に極めて重要になるので, 初学の段階でも意識しておくことをお勧めします

- 先ほどの問題を解くために必要な情報(入力情報)は, 「初項, 公差, 和に含まれる項数」

  ⇨ これらの入力情報をすべて変数としてプログラムを書いてみます

汎用性・可読性を高める 2/2

- 先ほどのプログラムに続けて, 右の6~13行目のプログラムを書いてみましょう

- for文の外で, 初項をinit, 公差をd, 和に含まれる項数をnum_termsという変数に格納します

- 10~13行目は, 以前のプログラム(1~4行目)と全く同じ構造ですが, 今定義された三つの変数によって書かれています

- 等差数列の詳細が変わっても, 6~8行目だけを変更すればよく, プログラムの主要部であるfor文以下のコードブロックは一切変更不要(汎用性)

```
1   total = 0
2   for i in range(10):
3       total += 3 + i*2
4   print(total)
5
6   init = 3
7   d = 2
8   num_terms = 10
9
10  total = 0
11  for i in range(num_terms):
12      total += init + i*d
13  print(total)
```

- 変数の名前を適切に設定することで, プログラムに書かれている処理の中身を, 人が読んで理解しやすくできる(可読性)

色々な等差数列の和を求めてみよう 1/2

・init, d, num_termsを自由に変更し, 色々な等差数列の和を計算してみましょう

・ただし, 出力された結果が本当に正しいかどうかをチェックする機能も追加しておきましょう

・今の問題設定では, 等比数列$a_n = a_0 + n*d$の初項(n=0)から第N項(n=N-1)までの和S_Nは$S_N = (N+1)*(2a_0 + N*d)/2$ と手で求められます

・そこで, 先ほどのプログラムに続けて, この公式で計算した和S_Nも出力するようにしておきましょう(具体的なプログラム例は次ページ)

  ・注1) この公式に含まれる「N」とプログラム上で保持している和に「含まれる項数num_terms」はnum_terms = N+1の関係にあります

  ・注2) 今の設定では$N \geq 0$なので, 注1)により$num_terms \geq 1$でないといけません

色々な等差数列の和を求めてみよう 2/2

- 前ページの注意点を踏まえると, 次のようなプログラムを書き足してS_Nの計算ができます

```
15    sol = 0.5*(num_terms)*(2*init+(num_terms-1)*d)
16    print("Solution by formula")
17    print(sol)
```

- 公式に従って計算したS_Nの結果を変数solに格納し, 最後にそれをprintします
  - プログラムの実行時に, どれが公式による結果か分かりやすいよう, 16行目のようなprint文を加えておきました
- init, d, num_termsを色々と変更し, for文の結果とprint(sol)の結果が一致するか確かめましょう

ifによる条件分岐 1/3

- 「ある条件が満たされた時だけ特定の処理をする」という手続きを条件分岐と呼びます
- 条件分岐はif文を使って書きます
  - ifに続いて, 真偽値を返す式(=条件式)を書き, 条件式の末尾に「：」を書く
  - 条件式がTrueの時だけ, それ以下に続くコードブロックが実行される
- work5の中に, 好きな名前で.pyファイルを用意し, 次のようなif文を書いてみましょう

```
1    if 1 < 10:
2        print("1<10")
```

- 実行してみましょう
  - 比較演算子「<」により, 1<10はTrueを返すため, 後続のprint文が実行されます

ifによる条件分岐 2/3

- 「条件が満たされる場合は処理Aを, そうでない場合は処理Bをする」という条件分岐は, ifとelseを組み合わせて実装します
- 先ほど書いたプログラムに続けて, 以下のif, elseからなるプログラムを書いてみよう

```
4    if 1 > 10:
5        print("A")
6    else:
7        print("B")
```

- 実行してみましょう
  - if文の1>10はFalseとなり, else側のコードブロックのみが実行されます
- elseの後ろには条件式がきません

ifによる条件分岐 3/3

- 「条件1が満たされる場合は処理Aを, そうでない場合で条件2が満たされる場合は処理Bをする」という条件分岐は, ifとelifを組み合わせて実装します
- 先ほど書いたプログラムに続けて, 以下のif, elifからなるプログラムを書いてみよう
- 実行してみましょう
  - 二つの変数x, yの大小に応じた条件分岐です
  - elifの後ろには条件式が必要です

```
 9    x = 1
10    y = 2
11    if x==y:
12        print("x=y")
13    elif x>y:
14        print("x>y")
15    else:
16        print("x<y")
```

条件分岐のネスト構造

・if文に続くコードブロックの中で, さらなる条件分岐を書くこともできる

　・コードブロックの中に別のコードブロックを記述することを「ネスト(入れ子)」と言う

・先ほど書いたプログラムに続けて, 次のようなプログラムを書いてみましょう

・実行してみましょう

　・x>10と出力されます

　・変数xの値を変え, 違う出力も得てみましょう

```
18    x = 12
19    if x > 5:
20        if x < 10:
21            print("5<x<10")
22        else:
23            print("x>10")
24    else:
25        print("x<=5")
```

if文を書くときの注意 1/2

- プログラム上で条件分岐を書く際には, 条件漏れがないように注意しましょう
- 条件漏れがあったからと言っていつでもエラーが生じる訳ではないですが, 思わぬエラーを生み出してしまう可能性があります
- どんな入力に対しても, 必ずいずれかのコードブロックが実行されるようなプログラムを書くように心がけましょう

if文を書くときの注意 2/2

- 具体例で見てみます

```python
a = 0

if a > 0:
    print("Positive")
elif a < 0:
    print("Negative")
```

```python
a = 0

if a > 0:
    print("Positive")
else:
    print("Not Positive")
```

- 左のコードでは, a=0の場合に実行されるコードブロックが存在しません
  - a=0の場合にはどちらのコードブロックもすり抜けてしまい, この下にさらにプログラムが続く場合には思わぬエラーを引き起こす可能性があります
- 右のコードでは, aにどんな値が来ても必ずいずれかのコードブロックが実行されます
  - if文やelif文を書く場合, else文を含めておくことで, 条件漏れを防止しやすくなります

whileによるループ処理 1/2

- 例えば,「ある条件が満たされている限り, 繰り返し処理を行う」というプログラムを書くことを考えます
- この時, プログラムを書く段階でループの反復回数が事前に分かる場合には, for文を使った実装で事足りるでしょう
- しかし, 場合によっては, プログラムを書く段階ではループの反復回数が何回になるのか分からない場合もあるでしょう
- このようなケースで役に立つのがwhile文です(while文による繰り返し処理もループと呼ばれます)
- 次ページで具体例を見てみましょう

whileによるループ処理 2/2

- work5の中に, 好きな名前で.pyファイルを用意し, 以下のwhile文によるプログラムを書き, 実行してみましょう

```
1    a = 10
2    while a>0:
3        print(a)
4        a = a-1
5        # a -= 1
```

複合演算子-=を使えば,
4行目は5行目のように
書くこともできます

- 元々, 変数aには10が代入されています

- while文に入ると, まずprint(a)が実行され, 続けてa-1を計算し, その結果で変数aの値が更新されます(= ループが回るごとにaの値が1だけ減る)

- while文に続く条件式a>0によって, 上記の処理がaが正である限り繰り返されます

ループ構文から抜ける方法

- for文やwhile文はループを作る構文であることを学びました
- ある条件が満たされた時に, ループをスキップしたい or ループを終了したいようなケースもあるでしょう
- 「ある条件が満たされた場合, ループをスキップする」とは「ある条件が満たされた場合, 何も処理を行わずループカウンタだけを1つ増やす」という意味である

  ⇒ continueを使う

- 「ある条件が満たされた場合, ループを終了する」

  ⇒ breakを使う

continueを使ったプログラムの例 1/2

- work5の中に, 好き名前で.pyファイルを用意し, まずは以下のような条件分岐を含むループを書いてみよう

```
1    for i in range(10):
2        if i%2 == 0:
3            print(i)
```

- 「a%b」はaをbで割った時の余りを返します
- 2行目以下のif文は, iを2で割った余りが0の場合(= iが偶数の場合), print(i)を実行する, という条件分岐になっています
- 実行し, 0~9の間に存在する偶数が出力されることを確認しましょう

continueを使ったプログラムの例 2/2

- 今度は先ほどのプログラムをcontinueを使って書いてみます
- 「ある条件が満たされた場合, ループをスキップする」という形に書き換えると…
  - 「 iが偶数の場合, print(i)を実行する」

  ⇨「 iが偶数ではない場合, ループをスキップして, print(i)を実行しない」

- 先ほどのプログラムに続け, continueを使って同じ内容のプログラムを書いてみましょう
  - 「a!=b」はaとbが異なる場合にTrueを返す比較演算子です
- 実行し, continueを使ったプログラムでもiが偶数の時だけprint(i)されることを確かめましょう

```
5    for i in range(10):
6        if i%2 != 0:
7            continue
8        print(i)
```

continueの使用を検討する価値があるケース

- 今見たように, 以下の二つのプログラムは同じ結果を与えます

```
1    for i in range(10):
2        if i%2 == 0:
3            print(i)
```

```
5    for i in range(10):
6        if i%2 != 0:
7            continue
8        print(i)
```

- このプログラムの主要部であるprint(i)は, 左ではif文のコードブロックに含まれますが, 右ではfor文のコードブロックに含まれています

- 一般に, プログラムの主要部を深いコードブロックに書くと, ネストされている構造を人が読んで理解するのに時間がかかる場合が多いです

- continueを使うことで, プログラムの主要部をより浅いコードブロックに持ってくることができ, プログラムの可読性の向上につながることがあります

breakを使ったプログラムの例 1/3

- 「ある条件が満たされた場合, ループを終了する」時にはbreakを使います

- work5の中に, 好き名前で.pyファイルを用意し, まずは以下のようなプログラムを書いてみましょう(プログラムの解説は次ページ)

```
1   import random
2
3   your_budget = 5
4
5   while True:
6       your_budget += random.randint(0,1)*2-1
7       print(your_budget)
8       if your_budget == 0:
9           print("Lose! You have run out of budget")
10          break
11      if your_budget == 10:
12          print("Success! Your budget has reached 10")
13          break
```

breakを使ったプログラムの例 2/3

- このプログラムはいわゆる無限ループで書かれています
  - whileに続く条件式にTrueを置いており, while文は条件式が満たされる限り処理を繰り返すので, 無限にループが回り続けます
- random.randint(0,1)は, 確率1/2で0か1の整数値(乱数)を返します
  - 1行目のimport文はこの乱数を使えるようにするため
- 変数your_budgetはあなたの手持ちの予算です
  - 6行目により, ループが回るごとに確率1/2で+1あるいは-1されます
- 手持ちの予算が0になるか, 10に達したら, breakによってループを終了させます

breakを使ったプログラムの例 3/3

- プログラムを実行し, 予算の変動と最終結果を見てみましょう
  - random.randint(0,1)は, プログラムの実行時ごとに確率1/2で0か1を返すので, このプログラムの結果は実行するごとに変わります
  - 何回か実行して遊んでみましょう

練習問題：Collatz問題 1/2

- Collatz (コラッツ) 問題とは「正の整数に対し, 次の操作を繰り返すと, どんな初期値から始めても最終的に1に到達する」という予想である
  - 正の整数nに対して, nが偶数ならばnを2で割り, nが奇数ならば3倍して1を足す
- 今のところこの予想の証明はないが, $n=2^{68}$までの初期値には反例が見つかっていないそうです

D. Barina, Convergence verification of the Collatz problem, J. Supercomput. 77, 2681-2688 (2021)

練習問題：Collatz問題 2/2

- Collatz問題をプログラムを書いて確認してみましょう
  - 初期値は自身で色々と変えやすいようにし, 予想を検証してみましょう
  - 正の整数nが1になるまで, 以下の二種類の処理を繰り返すようなプログラムを書いてみてください
    - nが偶数なら, n//2を計算し, その結果でnを更新する
    - nが奇数なら, 3*n+1を計算し,その結果でnを更新する
  - ループが回るごとに, 現在のnをprint(n)で出力し, n=1に到達するまでの履歴を見てみましょう

Hint)「nが1になるまで~を繰り返す」→ while n != 1:

初期値が3の場合の答え

- 例えば, 初期値として3を選んだ時の履歴は右のようになります
- 自分の得た出力と一致しているでしょうか？一致していればOK
- 他の初期値でも予想が成り立つか検証してみましょう

```
3
10
5
16
8
4
2
1
```

ここまで終わったら

・お疲れ様でした

・早く終わった人は, 第1回レポート課題に取り組んでください